

1. Basic Inheritance

Scenario: Design a superclass **Vehicle** with attributes **speed** (in km/h) and **color**. Create two subclasses: **Car** and **Bike**. Each subclass should override a method **displayInfo()** to print the type of vehicle, its speed, and color.

- **Sample Input:**
Car car = new Car(150, "Red");
Bike bike = new Bike(100, "Blue");
- **Sample Output:**
Car Speed: 150 km/h Color: Red
Bike Speed: 100 km/h Color: Blue

2. Constructor Chaining

Scenario: Implement a superclass **Person** with a parameterized constructor to initialize the person's name, and a subclass **Employee** that extends **Person**, adding a field **employeeId** and initializing it through its constructor.

- **Sample Input:**
Employee emp = new Employee("John Doe", "E100");
- **Sample Output:**
Name: John Doe, Employee ID: E100

3. Using super Keyword

Scenario: In a class **Animal** with a method **eat()** and a subclass **Dog** that overrides the **eat()** method. Use **super.eat()** in **Dog's eat** method to call the superclass method before adding additional behavior.

- **Sample Input:**
Dog dog = new Dog();
dog.eat();
- **Sample Output:**
Animal eats
Dog eats

4. Final Method

Scenario: Create a class **Shape** with a final method **draw()** that prints "Shape is drawn". Try to override this method in a subclass **Circle**.

- **Sample Input:**
Circle circle = new Circle();
circle.draw();
- **Sample Output:**
Shape is drawn

5. Method Overriding and Dynamic Binding

Scenario: Implement a superclass **Shape** with a method **area()** and subclasses **Circle** and **Square**. Each subclass overrides **area()** method. Demonstrate dynamic binding by calling **area()** on a **Shape** reference pointing to a **Circle** and **Square** object.

- **Sample Input:**
Shape shape1 = new Circle(5); // radius 5
Shape shape2 = new Square(4); // side 4
- **Sample Output:**
Circle area: 78.54
Square area: 16

6. Abstract Classes and Inheritance

Scenario: Define an abstract class **Animal** with an abstract method **makeSound()**. Create two subclasses **Dog** and **Cat** that implement **makeSound()**.

- **Sample Input:**
Animal dog = new Dog();
Animal cat = new Cat();
- **Sample Output:**
Dog barks
Cat meows

7. Polymorphism with Arrays

Scenario: Create an array of **Shape** objects where each element can be a **Circle**, **Square**, or **Triangle** (all subclasses of **Shape**). Loop through the array and call the **area()** method on each shape.

- **Sample Input:**
Shape[] shapes = {new Circle(3), new Square(5), new Triangle(4, 4)};
- **Sample Output:**
Circle area: 28.27

Square area: 25

Triangle area: 8

8. The final Keyword with Classes

Scenario: Attempt to extend a final class **ImmutablePoint** that has final **x** and **y** attributes initialized through its constructor.

- **Sample Input:**
ImmutablePoint point = new ImmutablePoint(1, 2);
- **Sample Output:**
Point created at (1,2)

9. Overriding vs. Overloading

Scenario: In a class **Printer**, demonstrate method overloading by creating multiple **print** methods that differ in their parameter list, and method overriding by extending the **Printer** class in a **ColorPrinter** subclass that overrides one of the **print** methods.

- **Sample Input:**
Printer printer = new ColorPrinter();
printer.print("Hello World");
printer.print("Hello World", "Blue");
- **Sample Output:**
Printing in Black & White: Hello World
Printing in Blue: Hello World

10. Interface Implementation

Scenario: Define an interface **Flyable** with a method **fly()**. Create two classes, **Bird** and **Airplane**, that implement **Flyable**. Demonstrate polymorphism by calling **fly()** on references of type **Flyable**.

- **Sample Input:**
Flyable f1 = new Bird();
Flyable f2 = new Airplane();
- **Sample Output:**
Bird flies flapping wings
Airplane flies with engines

11. Inheritance and Constructors

Scenario: Create a class **Book** with a constructor that initializes the book's title and a subclass **Ebook** that adds a field for the file size. Show how constructors are called when creating an instance of **Ebook**.

- **Sample Input:**
Ebook ebook = new Ebook("Java Fundamentals", 2.5);
- **Sample Output:**
Ebook created: Java Fundamentals, Size: 2.5MB

12. Encapsulation and Inheritance

Scenario: Design a class **Account** with private balance and a method to deposit money. Extend this class in **SavingsAccount** which adds an interest rate. Use getters and setters to access the private balance in the subclass.

- **Sample Input:**
SavingsAccount sa = new SavingsAccount(1000, 5);
sa.deposit(500);
sa.addInterest();
- **Sample Output:**
New Balance: 1575

13. Polymorphic Arguments

Scenario: Write a function in a class **Painter** that takes a **Shape** as an argument and calls the shape's **draw()** method. Pass instances of **Circle** and **Square** to this method.

- **Sample Input:**
Painter.paint(new Circle());
Painter.paint(new Square());
- **Sample Output:**
Drawing Circle
Drawing Square

14. Dynamic Method Dispatch

Scenario: Demonstrate dynamic method dispatch by creating a superclass reference to a subclass object. Use a **Vehicle** superclass and **Car** and **Bike** subclasses with overridden methods.

- **Sample Input:**
Vehicle v;
v = new Car();
v.display();
v = new Bike();
v.display();
- **Sample Output:**
Car display
Bike display

15. Using super in Method Overriding

Scenario: In a class hierarchy where **Superclass** has a method **display()** and **Subclass** overrides it. Use **super.display()** in the subclass to augment the method rather than replace its functionality.

- **Sample Input:**
Subclass subclass = new Subclass();
subclass.display();
- **Sample Output:**
Superclass display
Subclass display

16. Abstract Class with Final Method

Scenario: Create an abstract class **Device** with a final method **turnOn()** that prints "Device is on". Try to override this method in a subclass **Smartphone**.

- **Sample Input:**
Smartphone smartphone = new Smartphone();
smartphone.turnOn();
- **Sample Output:**
Device is on

17. Interface with Default Method

Scenario: Define an interface **Waterproof** with a default method **isWaterproof()** that returns **true**. Implement this interface in a class **Smartwatch** and override the default method if necessary.

- **Sample Input:**
Smartwatch smartwatch = new Smartwatch();
smartwatch.isWaterproof();
- **Sample Output:**
true

18. Multiple Interface Implementation

Scenario: Create two interfaces, **Flyable** and **Swimmable**, each with a method relevant to the action. Design a class **Duck** that implements both interfaces and provides implementation for the methods.

- **Sample Input:**
Duck duck = new Duck();
duck.fly();
duck.swim();
- **Sample Output:**
Duck flying
Duck swimming

19. Polymorphism in Collections

Scenario: Use a **List<Animal>** to store various animals like **Dog** and **Cat** (subclasses of **Animal**). Iterate over the list and call a method **speak()** on each animal.

- **Sample Input:**
List<Animal> animals = Arrays.asList(new Dog(), new Cat());
for (Animal animal : animals) {
animal.speak();
}
- **Sample Output:**
Dog barks
Cat meows

20. Final Class Extension Attempt

Scenario: Try to extend a final class **ImmutableClass** with a class **ExtendedClass**. Observe and explain the result.

- **Sample Input:**
// Attempt to extend a final class

- **Sample Output:**
Compilation error: Cannot extend a final class